

AGNAR RAKA BASKARA - 064102400014

Restful API di platform web

Pokok Bahasan

1. mahasiswa memahami cara kerja REST API di platform Web
2. mahasiswa mampu mengimplementasi rancangan akses REST API di platform web

Rincian Tugas Praktikum

1. Mahasiswa dapat merancang resource/sumberdaya API
2. Mahasiswa dapat mengimplementasikan rancangan aplikasi

Teori Singkat

API adalah komponen penting dalam pengembangan perangkat lunak modern, memungkinkan berbagai aplikasi dan layanan untuk berinteraksi secara efisien dan aman. Layanan API mencakup:

1. Metode (Fungsi/End Points): Fungsi atau prosedur yang dapat digunakan oleh aplikasi klien.
2. Data Format: Format data yang dipertukarkan, misalnya JSON, XML.
3. Protokol: Protokol yang digunakan untuk komunikasi, seperti HTTP/HTTPS.
4. Autentikasi: Mekanisme untuk memastikan bahwa hanya pengguna atau aplikasi yang berwenang yang dapat mengakses API.

1. Jenis-jenis API

API dapat dikategorikan berdasarkan siapa yang dapat mengakses API tersebut.

- **API Publik (Open API):** Dapat diakses oleh siapa saja. Contohnya adalah API untuk layanan cuaca atau API Google Maps. Berbagai API layanan cuaca dapat diakses <https://free-apis.github.io/#/categories/Weather>. Layanan API ini ada yang bisa diakses free atau berbayar.
- **API Privat:** Hanya dapat diakses oleh sistem internal organisasi. Misalnya, API untuk mengakses database internal perusahaan. Contoh, sebuah perusahaan listrik tenaga uap memanfaatkan Sistem Informasi Batubara untuk mengelola batubara di tongkang dan coalyard. Informasi tersebut mencakup karakteristik batu bara yang digunakan seperti kandungan karbon, kelembaban, dll. Perusahaan ingin mengembangkan sebuah aplikasi untuk melakukan blending (pencampuran) batu bara dari kategori berbeda. Untuk itu diperlukan data input berupa ketersediaan batubara yang akan digunakan dari setiap kategori. Dua aplikasi ini dapat berkomunikasi menggunakan API privat.

- **API Mitra:** Dibagikan dengan mitra bisnis tertentu. Biasanya memerlukan autentikasi dan izin khusus. Contoh misalnya traveloka bekerja sama dengan blue bird untuk mengembangkan aplikasi penyewaan taksi bandara. Pihak blue bird membuka layanan akses ke API sumberdaya yang diperlukan oleh traveloka sehingga traveloka dapat menggunakannya pada aplikasi yang mereka kembangkan melalui akses lewat API.

2. Keuntungan Menggunakan API

- **Integrasi Mudah:** Memudahkan integrasi antara berbagai aplikasi dan layanan.
- **Efisiensi Pengembangan:** Mengurangi waktu pengembangan dengan memanfaatkan fungsi yang sudah ada.
- **Interoperabilitas:** Memungkinkan berbagai sistem untuk bekerja sama meskipun dibangun dengan teknologi yang berbeda.
- **Keamanan:** API dapat membantu mengontrol akses dan menjaga keamanan data.

3. Contoh Penggunaan API

- **Media Sosial:** API Facebook atau Twitter memungkinkan aplikasi pihak ketiga untuk memposting status atau membaca data pengguna.
- **Pembayaran Online:** API PayPal atau Stripe memungkinkan integrasi pembayaran dalam aplikasi e-commerce.
- **Layanan Geolokasi:** API Google Maps memungkinkan aplikasi untuk menampilkan peta dan mendapatkan data lokasi.

4. Cara Kerja API

- **Request dan Response:** Aplikasi klien mengirimkan permintaan (request) ke API server. Server memproses permintaan dan mengirimkan respons kembali ke klien.
- **RESTful API:** Salah satu jenis API yang paling umum, menggunakan HTTP untuk berkomunikasi dan biasanya mengembalikan data dalam format JSON atau XML.
- **SOAP API:** Protokol berbasis XML yang digunakan untuk pertukaran informasi terstruktur dalam implementasi web services.

5. Keamanan API

- **Token Autentikasi:** Menggunakan token seperti JWT (JSON Web Token) untuk mengamankan komunikasi.
- **SSL/TLS:** Menggunakan protokol keamanan untuk enkripsi data yang dipertukarkan.
- **Rate Limiting:** Membatasi jumlah permintaan yang dapat dilakukan dalam periode waktu tertentu untuk mencegah penyalahgunaan.

Contoh pemanfaatan API pada pengelolaan batubara di PLTU

Untuk melakukan optimasi pemanfaatan batubara dan memenuhi regulasi pemerintah terkait kelistrikan, sebuah perusahaan PLTU harus melakukan pencampuran dua batubara dari kategori berbeda dengan biomassa sesuai ketersediaan batubara yang ada ditongkang dan di coalyard. Sistem Informasi pengelolaan batubara sudah mereka miliki. Mereka perlu menyediakan akses bagi sistem baru yang akan mereka kembangkan ke database Sistem Informasi pengelolaan batubara. Penyediaan akses melalui API memudahkan integrasi antara kedua sistem tersebut. Tahapan implementasi dari aplikasi yang dibuat:

1. Identifikasi kebutuhan data pada sistem baru
2. Ketersediaan data dari Sistem Informasi Pengelolaan Batubara
3. Desain API
4. Implementasi Sistem.

1. Tahap Identifikasi kebutuhan data pada sistem baru

Prosedur Blending batubara pada PLTU adalah melakukan blending dua kategori berbeda yang berasal dari tongkang dan dari coalyard. Jika di tongkang tersedia batubara LRC maka dari coalyard akan dipilih MRC yang optimal. Jika di tongkang MRC maka yang diambil dari coalyard adalah LRC. Perhitungan untuk menentukan mana komposisi optimal dilakukan di di sistem baru dengan memanfaatkan Algoritma Genetik. Pada bagian ini, kita tidak akan membahas hal tersebut.



2. Ketersediaan data dr Sistem Informasi Pengelolaan Batubara

Untuk mendapatkan informasi terkait karakteristik dan lokasi batubara yang ada di tongkang dan dicoalyard, maka sistem informasi pengelolaan batubara akan menyediakan data yang berasal dari hasil queri kedalam database yang mencakup sejumlah tabel didalam database. Lebih jelasnya deskripsi dari setiap kategori batubara dinyatakan dalam format json berikut:

```
{
  "id": 1,
  "sumberpasokan": "coalyard",
  "Pemasok": "Bukit Asam",
  "Kategori": "batubara mrc",
  "Kalori": 4917,
  "TM": 29.04,
  "TS": 0.36,
  "ASH": 5.82,
  "IDTReducing": 1256,
  "SiO2": 58.1,
  "Al2O3": 25.4,
  "Fe2O3": 4.93,
  "CaO": 3.19,
  "MgO": 1.56,
  "Na2O": 1.45,
  "K2O": 0.76,
  "TiO2": 0.7,
  "SO3": 3.2,
  "HTmax": 14000,
  "JenisAbu": "Bituminous",
  "SI": 0.05,
  "Slagging": "low",
  "C": 61.26,
  "H": 4.44,
  "N": 0.94,
  "O": 14.28,
  "statusalat": 1
}
```

3. Desain API

Untuk menyajikan informasi/sumberdaya yang tersedia, kita bisa memanfaatkan API contract yang memuat minimal behavior dari API, endpointnya, dan bagaimana API digunakan. Contoh pada API cuaca berikut:

```
GET /coalstockfromcoalyard
```

Input parameters:

- none

Output:

- sumber pasokan: pasokan dipastikan berasal dari coalyard
- pemasok : suplier yg memasok batubara
- kategori batubara : ada dua kategori batubara LRC dan MRC
- karakteristik : meliputi kandungan kalori, TM, TS, ASH, dst seperti pada uraian nomor 2

Contoh penggunaan:

```
GET /coalstockfromcoalyard
```

Output:

Seperti disajikan pada Gambar diatas.

Tahapan desain secara lengkap mengacu kepada [praveen] adalah sebagai berikut:

1. Identifikasi sumber daya (tuliskan dalam bentuk jamak misal students bukan student)
2. Definisikan model dari sumberdaya dan mendefinisikan identifier dari masing-masing model
3. Mendesain hubungan antar resource
4. Pengecekan kompleksitas URL
5. Mendesain operasi API setiap resource
6. Mendesain request dan respons API
7. Desain filtering, pagination dan sorting

4. Implementasi Sistem

1. Mendefinisikan class model untuk merepresentasikan data yang diperlukan

```
1  from django.db import models
2
3  # Create your models here.
4  class CoalStockFromCoalyard(models.Model):
5      locationId=models.IntegerField()
6      level=models.IntegerField()
7      sumberpasokan=models.CharField(default='coalyard',max_length=15)
8      Pemasok = models.CharField(max_length=100)
9      Kategori = models.CharField(max_length=15)
10     Kalori = models.IntegerField()
11     TM = models.FloatField(max_length=8)
12     TS = models.FloatField(max_length=8)
13     ASH = models.FloatField(max_length=8)
14     IDTReducing = models.IntegerField()
15     SiO2 = models.FloatField(max_length=8)
16     Al2O3 = models.FloatField(max_length=8)
17     Fe2O3 = models.FloatField(max_length=8)
18     CaO = models.FloatField(max_length=8)
```

2. Mendefinisikan serializer dari setiap kelas turunan dari **serializers.ModelSerializer**

```

1  from .models import CoalStockFromCoalyard, CoalStockFromTongkang, CoalStock
2  from rest_framework import serializers
3
4  class CoalStockFromCoalyardSerializer(serializers.ModelSerializer):
5      class Meta:
6          model = CoalStockFromCoalyard
7          exclude = ['locationId', 'level']
8
9  class CoalStockFromTongkangSerializer(serializers.ModelSerializer):
10     class Meta:
11         model = CoalStockFromTongkang
12         fields = '__all__'
13

```

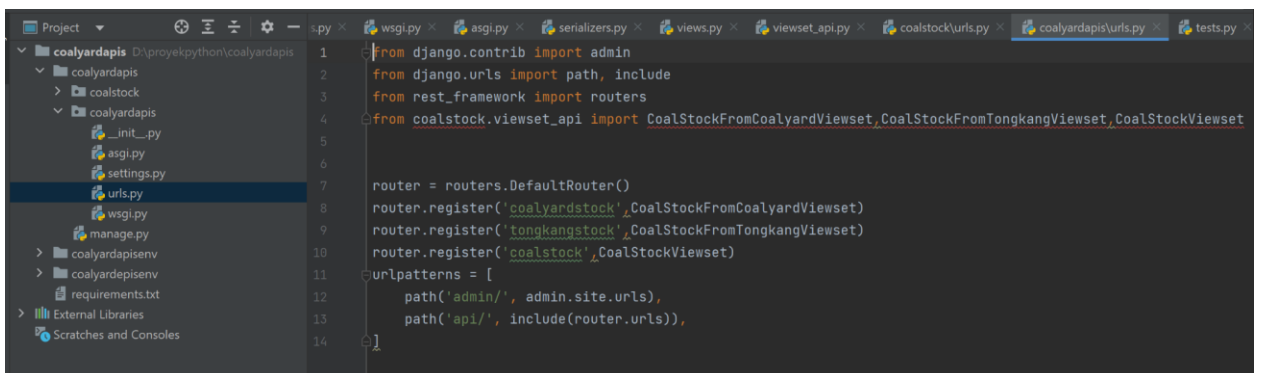
3. Membuat class turunan dari viewsets.ModelViewSet di viewset.py untuk mendefinisikan **queryset**, **serializer_class**, **authentication_classes**, **permissions_classes**

```

1  from .models import CoalStockFromTongkang, CoalStockFromCoalyard, CoalStock
2  from .serializers import CoalStockFromCoalyardSerializer, CoalStockFromTongkangSerializer, CoalStockSerializer
3  from rest_framework import viewsets, permissions
4  from rest_framework.authentication import TokenAuthentication
5
6  class CoalStockFromCoalyardViewSet(viewsets.ModelViewSet):
7      queryset = CoalStockFromCoalyard.objects.all()
8      serializer_class = CoalStockFromCoalyardSerializer
9      authentication_classes = (TokenAuthentication,)
10     permissions_classes = (permissions.IsAuthenticated,)
11
12     class CoalStockFromTongkangViewSet(viewsets.ModelViewSet):
13         queryset = CoalStockFromTongkang.objects.all()
14         serializer_class = CoalStockFromTongkangSerializer
15         authentication_classes = (TokenAuthentication,)
16         permissions_classes = (permissions.IsAuthenticated,)
17

```

4. Mendefinisikan reuter dan mendaftarkan ke url.py



The screenshot shows a Django project named 'coalyardapis' with a file explorer on the left. The file explorer shows the project structure, including 'coalyardapis', 'coalyardapisenv', 'requirements.txt', and 'External Libraries'. The main editor shows the 'urls.py' file, which contains the following code:

```

1  from django.contrib import admin
2  from django.urls import path, include
3  from rest_framework import routers
4  from coalstock.viewset_api import CoalStockFromCoalyardViewSet, CoalStockFromTongkangViewSet, CoalStockViewSet
5
6
7  router = routers.DefaultRouter()
8  router.register('coalyardstock', CoalStockFromCoalyardViewSet)
9  router.register('tongkangstock', CoalStockFromTongkangViewSet)
10 router.register('coalstock', CoalStockViewSet)
11
12 urlpatterns = [
13     path('admin/', admin.site.urls),
14     path('api/', include(router.urls)),
15 ]

```

Konsumsi Restfull API:

1. Inputkan endpoint.
2. Definisikan fungsi untuk mengambil data dari endpoint, gunakan fungsi `response.get(url)`
3. Ubah data kedalam format json dengan `json.loads()`

```
if request.method == "POST":
    form = SetURLAddressForm(request.POST)
    if form.is_valid():
        curl = form.cleaned_data['coalyardurl']
        turl = form.cleaned_data['tongkangurl']
        unit = form.cleaned_data['unit']
        tanggal = datetime.date.today()
        coalyardresponse = requests.get(curl)
        tongkangresponse = requests.get(turl)
        coalyards = json.loads(coalyardresponse.text)
        tongkang = json.loads(tongkangresponse.text)
```

Penjelasan pada contoh kasus diatas:

- (1) endpoint diambil dari form field `coalyardurl` dan `tongkangurl`.
- (2) Untuk mengambil data yang tersedia di endpoint, gunakan `request.get(url)`
- (3) Untuk mengekstrak datanya dan mengubah kedalam format json dapat digunakan fungsi `json.load` pada variabel yang digunakan untuk menyimpan hasil `get`.

Tugas Praktikum.

Gunakan modul yang telah diselesaikan pada praktikum sebelumnya

Tahapan:

1. Mendefinisikan serializer dari class Content di file serializers.py

```
data_entry > serializers.py > ContentSerializer
1 from .models import Content
2 from rest_framework import serializers
3
4 class ContentSerializer(serializers.ModelSerializer):
5     class Meta:
6         model = Content
7         fields = '__all__'
```

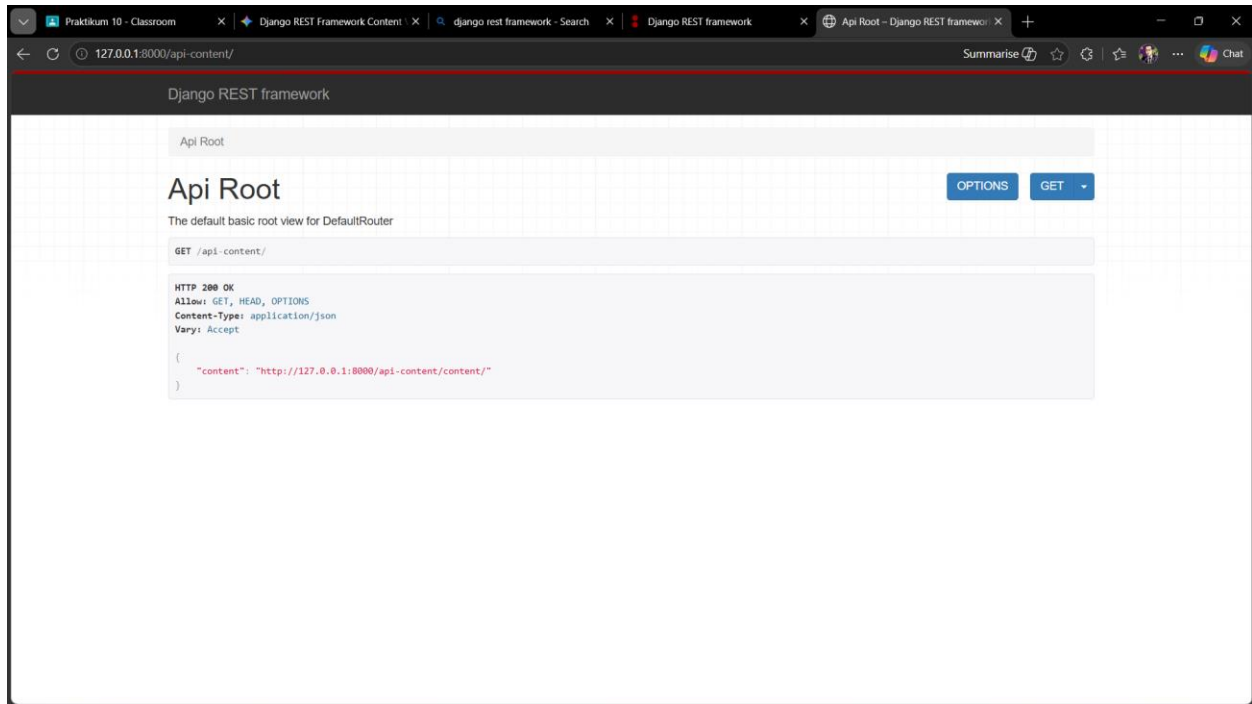
2. Mendefinisikan viewset_api

```
data_entry > viewset_api.py > ...
1 from .models import Content
2 from .serializers import ContentSerializer
3 from rest_framework import viewsets, permissions
4 from rest_framework.authentication import TokenAuthentication
5
6 class ContentViewSet(viewsets.ModelViewSet):
7     queryset = Content.objects.all()
8     serializer_class = ContentSerializer
9     authentication_classes = (TokenAuthentication,)
10    permissions_classes = (permissions.IsAuthenticated,)
```

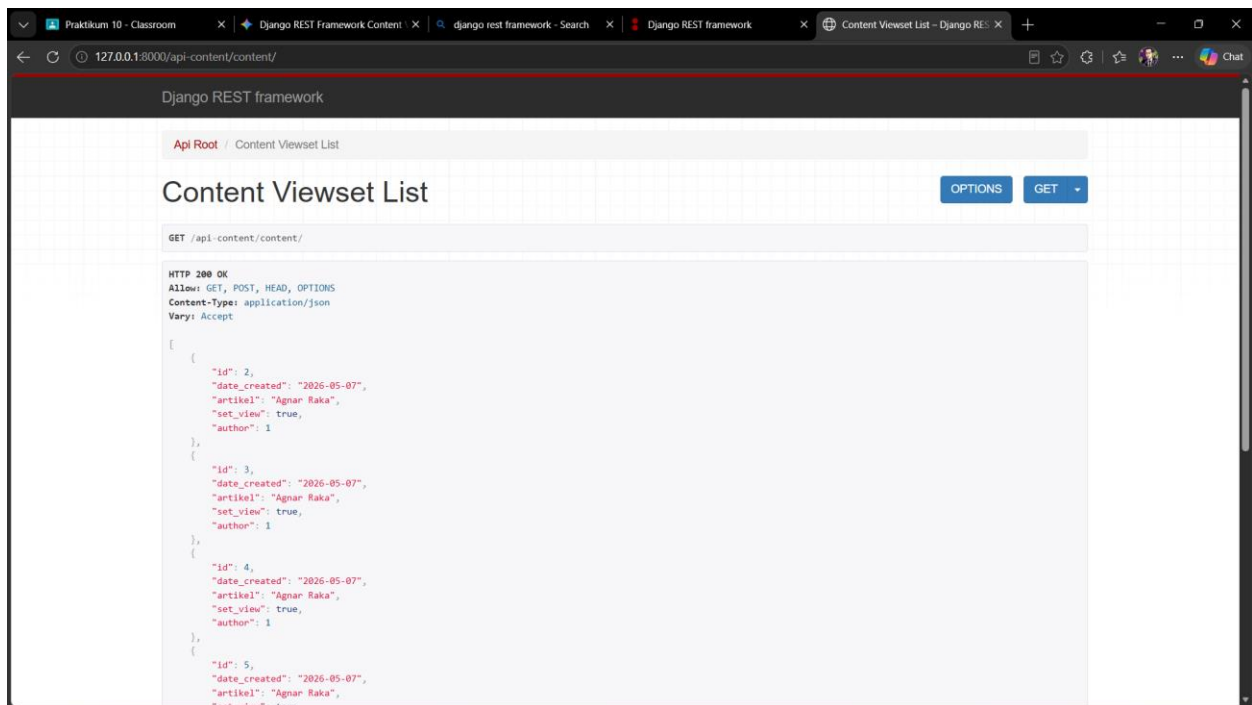
3. Mendaftarkan viewset ke reuter di file urls.py di folder proyek

```
1 """
2 URL configuration for menej_class_sistem_cerdas project.
3
4 The 'urlpatterns' list routes URLs to views. For more information please see:
5     https://docs.djangoproject.com/en/6.0/topics/http/urls/
6 Examples:
7 Function views
8     1. Add an import: from my_app import views
9     2. Add a URL to urlpatterns: path('', views.home, name='home')
10 Class-based views
11     1. Add an import: from other_app.views import Home
12     2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
13 Including another URLconf
14     1. Import the include() function: from django.urls import include, path
15     2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16 """
17 from django.contrib import admin
18 from django.urls import path, include
19 from rest_framework import routers
20 from data_entry.viewset_api import ContentViewSet
21
22 router = routers.DefaultRouter()
23 router.register('content', ContentViewSet)
24 app_name = 'data_entry'
25
26 urlpatterns = [
27     path('admin/', admin.site.urls),
28     path('', include(('data_entry.urls', 'data_entry'), namespace='data_entry')),
29     path('api-content/', include(router.urls)),
30 ]
31
```


4. Menjalankan aplikasi dan cek endpoint. Gunakan python manage.py runserver. Masukkan url ke url browser dan anda menemukan tampilan berikut.



Untuk cek datanya klik di url disisi kanan content (tulisan berwarna merah)



Anda sudah berhasil menyajikan data melalui endpoint.

Tahap selanjutnya, kita akan mempraktekan bagaimana mengkonsumsi data yang ada di endpoint

Praktikum Pengambilan data melalui API

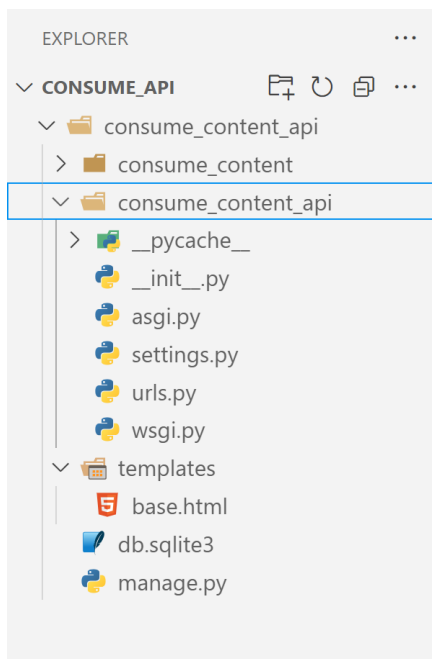
Konsumsi API dari web dilakukan dengan mendefinisikan fungsi untuk melakukan pengambilan data. Pada praktikum ini kita akan demonstrasikan bagaimana memanfaatkan data dari API dan menampilkan di web.

1. Buat Folder project_rest_api
2. Buat virtual environment baru bernama consume_api_env
3. Buat Project Baru bernama consume_content_api

gunakan perintah :

```
django-admin startproject consume_content_api
```

Struktur direktori proyek consume_content_api terlihat dari Gambar berikut:

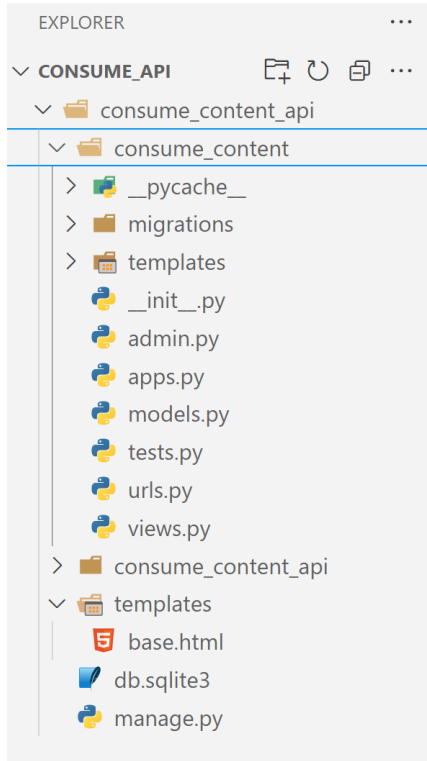


4. Buat App baru bernama consume_content

Gunakan perintah

```
django-admin startapp consume_content
```

Struktur direktori setelah create app:



5. Buat [urls.py](#) untuk mendefinisikan url baru untuk menampilkan content

Tambahkan path

```
path('content/', views.show_all_content, name = show_content),
```

6. Definisikan sebuah fungsi view pada [views.py](#) untuk menampilkan content dari API

```
def show_all_content(request):  
    # kerjakan disini  
    # 1. Tentukan url yang akan diakses  
    # 2. Panggil fungsi request.get(url)  
    # 3. Load sebagai json  
    # 4. Kirimkan sebagai content didalam context
```

7. Pastikan Aplikasi penyedia API masih berjalan.
8. Jalankan aplikasi pada port yang berbeda misalnya 8001 atau 5000

```
python manage.py runserver 0.0.0.0:<your_port>
```

Referensi:

1. Designing Restfull API
2. API Design Document Template
3. Ref: <https://bump.sh/blog/api-architecture-diagrams>